

LearnWD 代码说明文档

适用版本：当前仓库 (Python 3.14 + numpy 2.4 + scikit-learn 1.8)

运行环境：`.venv/` 虚拟环境，所有命令使用 `.venv/bin/python3`

目录

1. 项目结构总览
2. `config.py` — 全局常量
3. `pcm_sim.py` — PCM 物理模型
4. `stale_table.py` — Stale Table 数据结构
5. `encoding.py` — 写编码方案
6. `learnwd.py` — LearnWD 四组件模型
7. `selector.py` — Stale Block 选择策略
8. `simulation.py` — 主仿真循环
9. `data_loader.py` — 实验数据加载
10. `download_datasets.py` — 全量数据集下载/生成
11. `experiments.py` — 灵敏度实验实现
12. `run.py` — Exp#1-5 入口
13. `run_all.py` — 全量 15 数据集实验
14. `run_sensitivity.py` — Exp#6-12 灵敏度实验入口
15. 如何运行各实验
16. 修改代码后如何重新运行

1. 项目结构总览

```
learnWD/
├── config.py           # PCM 硬件参数常量 (SPEC 标准值)
├── pcm_sim.py          # PCM 写操作物理建模: WD-prone / 能耗 / 时延
├── stale_table.py      # Stale Table: 存储 stale block 地址 + 聚类 + MinHash
├── encoding.py         # 4 种写编码: DCW / DIN / DMPart / MinWD
├── learnwd.py          # LearnWD 四组件模型核心实现
├── selector.py         # Stale block 选择策略: RandSel / LearnWD 工厂
├── simulation.py       # 主仿真循环 (单次完整写工作负载)
├── data_loader.py      # 实验用数据加载 (MNIST / 合成数据 / raw 文件)
├── download_datasets.py # 全量 16 个真实数据集下载 → datasets/*.npz
├── experiments.py      # Exp#6-12 灵敏度实验函数库
├── run.py              # Exp#1-5:8 配置对比 (单数据集)
├── run_all.py          # 全量 15/16 数据集 × 8 配置主实验
├── run_sensitivity.py  # Exp#6-12 灵敏度实验入口
├── datasets/           # 预处理后的 .npz 文件 + manifest.json + kensho.zip
└── RESULTS.md         # 实验结果报告
```

模块依赖关系

```
config.py
↓
pcm_sim.py ← stale_table.py ← learnwd.py
      ↓           ↓           ↓
      encoding.py selector.py
      ↓           ↓
      simulation.py
      ↓
data_loader.py / download_datasets.py
      ↓
run.py / run_all.py / run_sensitivity.py
```

2. config.py — 全局常量

功能：集中定义所有 PCM 硬件参数和 LearnWD 默认超参数，所有模块通过 `from config import ...` 引用。修改此文件后，无需修改其他文件即可全局生效。

关键常量说明

```
BLOCK_BITS = 512          # PCM block 大小 (bits), 对应 64 字节 cacheline
STALE_POOL_SIZE = 50_000  # Stale Pool 容量 (block 数量)
WRITE_REQ_SIZE = 50_000  # 每次实验的写请求总数

# SLC PCM 单元错误率 (来自 SPEC Table II)
SLC_WD_WORDLINE = 0.099   # 每个 WD-prone 单元因字线扰动出错的概率
SLC_WD_BITLINE = 0.115   # 每个 WD-prone 单元因位线扰动出错的概率
# 注: 一个 WD-prone 单元的总出错概率  $\approx 0.099 + 0.115 = 0.214$  (两路独立扰动叠加)

SLC_RESET_LATENCY = 100   # 每个 RESET 写操作 (1→0) 的延迟 / ns
SLC_SET_LATENCY = 200     # 每个 SET 写操作 (0→1) 的延迟 / ns
SLC_RESET_ENERGY = 0.0137 # 每个 RESET 的能耗 / nJ
SLC_SET_ENERGY = 0.0268  # 每个 SET 的能耗 / nJ
RESET_COST_WEIGHT = 2     # RESET 代价权重 (写代价 =  $2 \times \text{RESET} + 1 \times \text{SET}$ )

DEFAULT_K = 16            # LearnWD k-means 聚类数
DEFAULT_H = 8             # MinHash 哈希函数数量
MINHASH_BITS = 3          # 每个 MinHash 值保留的位数 (掩码 =  $0b111 = 7$ )
RETRAIN_INTERVAL = 20_000 # 每隔多少次写操作触发一次 LearnWD 重训练
```

3. pcm_sim.py — PCM 物理模型

功能：对单次 block 写操作进行物理建模，计算 WD-prone 计数、WD 错误采样、写延迟、写能耗、写代价。

核心函数

```
compute_wd_prone(new_block, stale_block, write_mask) → int
```

最重要的函数。计算一次写操作产生的 WD-prone 相邻单元数。

物理原理：

- 在 PCM 中，RESET 操作（写 1→0）会向字线/位线方向发射热量，若相邻单元当前处于 idle-zero 状态（存

储 0 且本次不被写入)，则有概率因热扰动发生误写，即 Write Disturbance (WD)。

- RESET 位置 = `write_mask[i]=1` 且 `new_block[i]=0` (即要写 0 且之前是 1)

- Idle-zero 邻居 = `stale_block[j]=0` 且 `write_mask[j]=0` (存储 0 且不被写)

- WD-prone 数量 = Σ (每个 RESET 位置左右各一个 idle-zero 邻居的存在情况)

```
def compute_wd_prone(new_block, stale_block, write_mask):
    resets = (write_mask == 1) & (new_block == 0) # RESET 位置
    idle_zero = (stale_block == 0) & (write_mask == 0) # idle-zero 位置
    left_contrib[1:] = idle_zero[:-1] # 左邻居是否 idle-zero
    right_contrib[:-1] = idle_zero[1:] # 右邻居是否 idle-zero
    return sum(resets * (left_contrib + right_contrib)) # 每个 RESET 贡献 0/1/2
```

`simulate_wd_errors(wd_prone_count, rng) → int`

从 WD-prone 计数采样实际 WD 错误数：

```
wordline_errors = rng.binomial(wd_prone_count, SLC_WD_WORDLINE) # 字线方向
bitline_errors = rng.binomial(wd_prone_count, SLC_WD_BITLINE) # 位线方向
return wordline_errors + bitline_errors
```

两路独立二项采样，符合 SLC PCM 的双向扰动机制。

`compute_write_cost / compute_write_latency / compute_write_energy`

```
# 写代价 (无单位, 用于比较)
cost = reset_count * RESET_COST_WEIGHT + set_count * SET_COST_WEIGHT # = 2R + S

# 写延迟 (ns)
latency = reset_count * SLC_RESET_LATENCY + set_count * SLC_SET_LATENCY
          + vnr_count * (SLC_RESET_LATENCY + VNR_VERIFY_LATENCY) # VnR 额外开销

# 写能耗 (nJ)
energy = reset_count * SLC_RESET_ENERGY + set_count * SLC_SET_ENERGY
```

4. stale_table.py — Stale Table 数据结构

功能：维护 Stale Pool 的所有 block 的地址、聚类 ID 和 MinHash 指纹，支持 O(1) 按簇查询。

数据结构设计

```
StaleTable
├─ _entries: dict[int, StaleEntry] # 物理地址 → 条目
├─ _cluster_idx: dict[int, set[int]] # 聚类ID → 地址集合 (倒排索引)
```

`_cluster_idx` 是核心优化：“LearnWD 需要频繁查询”某个 cluster 中所有 blocks”，若遍历 `_entries` 则 O(N)=O(50000)；倒排索引将其降为 O(cluster_size)。

关键方法

方法	复杂度	说明
<code>insert(address, cluster_id, minhash_values)</code>	O(1)	插入新 stale block
<code>delete(address)</code>	O(1)	删除 (block 被写入后)

方法	复杂度	说明
<code>query_by_cluster(cluster_id)</code>	$O(M_c)$	返回该簇所有 StaleEntry, M_c 为簇大小
<code>update_cluster(address, cluster_id, minhash)</code>	$O(1)$	LearnWD 重训后更新簇分配
<code>bulk_insert(addresses)</code>	$O(N)$	初始化时批量插入

5. encoding.py — 写编码方案

功能：定义 4 种编码器，每种接受 `(new_block, stale_block)` 返回 `(encoded_block, write_mask)`。

编码器接口

```
def encoder(new_block: np.ndarray,          # (512,) uint8, 待写入数据
            stale_block: np.ndarray        # (512,) uint8, 当前 stale block
) -> tuple[np.ndarray, np.ndarray]:        # (encoded_block, write_mask)
    # write_mask = encoded_block XOR stale_block, 即实际写入的位
```

四种编码器详解

`dcw_encode` — Data Comparison Write (基准)

```
write_mask = new_block XOR stale_block    # 只写变化的位
return new_block, write_mask
```

最简单，只写实际改变的位，不修改 `new_block` 内容。

`dmpart_encode` — Data Manipulation by Partitioning

1. 将 512 bits 分成 256 个 2-bit 分区
2. 统计 4 种 2-bit 模式 (00/01/10/11) 的出现频率
3. 选出出现最少的模式作为 XOR 掩码
4. 所有分区与掩码 XOR → 最少出现的模式变为 `00`，减少 RESET 操作

```
mask_int = argmin(bincount(pattern_idx))    # 出现最少的模式
encoded = parts XOR mask_bits               # 将最少模式映射为 00
```

代价：需要 2-bit side-channel metadata 存储掩码 (excluded from block)。

`minwd_encode` — Minimum WD-prone

枚举 4 种变换，选 WD-prone 计数最小的：

变换索引	变换内容
0	原始 <code>new_block</code>
1	<code>~new_block</code> (全位取反)
2	前后半段互换 <code>[256:]:[0:256]</code>
3	半段互换后再取反

```

candidates = [original, inverted, swapped, swapped+inverted]
scores = _wd_prone_batch(candidates, stale_block) # 向量化计算4路WD-prone
best = argmin(scores)

```

`_wd_prone_batch` 使用向量化 numpy 同时计算 4 种候选的 WD-prone 数，无 Python 循环。

`din_encode` — Data-Informed Narrow (FPC + BCH)

1. FPC 压缩 (`_fpc_compress`): 将 512 bits (16 个 32-bit 字) 用 FPC 压缩
 - FPC (Frequent Pattern Compression) 按优先级匹配 8 种常见 32-bit 数据模式
 - 压缩后产生可变长度 bit 流 (大量零值数据压缩率很高)

2. 若压缩后 ≤ 492 bits:

```
encoded_block = [FPC数据 | 零填充(到492位) | BCH-20校验码]
```

零填充区域全部为 0，若 stale block 对应位也是 0 则无 RESET，大幅减少 WD

3. 若压缩后 > 492 bits: 回退到 `dcw_encode`

`_bch20`: 20-bit 简化奇偶校验 (Exp#6 VnR 开销仿真用，非正式 BCH 实现)。

6. learnwd.py — LearnWD 四组件模型

功能：实现 LearnWD 智能 stale block 选择的四个组件。

四组件架构

```

新写请求 new_block
↓
① extract_aggressor(new_block) → aggressor_vector (512,)
↓
② select_cluster(aggressor) → cluster_id (0..k-1)
↓
③ estimate_similarity(new_block, cluster_id, stale_table) → stale_addr
↓
选出的 stale_addr

```

另有训练组件：

```

train(stale_memory, stale_table) 每 RETRAIN_INTERVAL 写操作调用一次
→ ① 计算所有 stale block 的 disturbance vector
→ ② k-means 聚类
→ ③ MinHash 指纹计算
→ ④ 更新 StaleTable 条目 + 重建缓存

```

组件详解

- ① `extract_aggressor(new_block) → np.ndarray`

```

# 标记"潜在 RESET 攻击位": new_block=0 且至少有一个 0 邻居的位置
zeros = new_block == 0
left_zero = np.roll(zeros, 1); left_zero[0] = False
right_zero = np.roll(zeros, -1); right_zero[-1] = False
aggressor = zeros & (left_zero | right_zero)

```

直觉：若 new_block 的某一位是 0 且邻居也是 0，说明该位置写 1→0（若 stale 是 1）会产生 WD-prone 邻居。

② `select_cluster(aggessor) → int`

```
scores = self.centroids @ aggessor.astype(float32)  # (k,)
return argmin(scores)
```

选 dot(aggessor, centroid) 最小的簇。直觉：centroid 代表该簇 blocks 在各位置的平均“WD 危险度”；dot 积越小说明 stale blocks 的 WD 热点与 new_block 的 RESET 位置越不重叠，WD-prone 越少。

③ `estimate_similarity(...) → int`

```
new_hash = _minhash_single(new_block)  # (h,) uint8
sims = sum(c_hashes == new_hash, axis=1)  # 按位匹配计数
best = argmax(sims)  # 最相似的 stale block
return c_addrs[best]
```

MinHash 近似 Jaccard 相似度：匹配的 hash 值越多，两个 block 的 1-bit 集合的 Jaccard 相似度越高。找 Jaccard 相似度最高的 stale block 用于写入（其 1-bits 与 new_block 最接近，差分写开销最小，同时也使得 WD-prone 较低）。

`train()` — 模型训练

```
# 1. 计算所有 stale block 的 disturbance vector
#   d[i] = (block[i]==1) * (左邻居==0 的数量 + 右邻居==0 的数量)
#   值域 {0, 1, 2}, 表示该 1-bit 旁边有几个 0-bit 邻居 (WD 危险度)
dist_vecs = _disturbance_batch(blocks)  # (N, 512) float32

# 2. k-means / GMM / BIRCH 聚类
cluster_ids, centroids = _cluster(dist_vecs)

# 3. MinHash 指纹 (向量化)
minhashes = _minhash_batch(blocks)  # (N, h) uint8

# 4. 更新 StaleTable 条目 (聚类ID + 指纹)
# 5. 重建缓存 (避免后续 O(N) 扫描)
```

`invalidate(address)` — O(1) 缓存失效

```
c, row = self._addr_pos[address]
self._c_valid[c][row] = False  # 该 block 已被覆写, 从有效集合移除
```

每次 stale block 被写入后调用。通过预建的 `_addr_pos` dict 直接定位，O(1)。

7. selector.py — Stale Block 选择策略

功能：定义两种选择策略，统一接口。

接口约定

```
def selector(
    new_block: np.ndarray,      # 待写入 block
    stale_table: StaleTable,    # Stale Table 对象
    stale_memory: np.ndarray,    # Stale Pool 内存 ((N, 512) uint8)
    rng: np.random.Generator,
    **kwargs,
) -> int:                      # 返回选中的物理地址
```

randssel

```
addresses = stale_table.all_addresses()
return addresses[rng.integers(0, len(addresses))] # 均匀随机
```

make_learnwd_selector(model) — 工厂函数

返回一个包装了 `model.select()` 的 `selector` 函数。用工厂模式而非类，使 `selector` 与 `simulation` 接口解耦。

```
def _select(new_block, stale_table, stale_memory, rng, **_):
    return model.select(new_block, stale_table, stale_memory, rng)
return _select
```

8. simulation.py — 主仿真循环

功能：执行单次完整写工作负载（`write_requests` 中每个 block 依次写入），记录每步指标。

run_simulation() 参数

```
def run_simulation(
    stale_blocks,      # (N_stale, 512) uint8 - 初始 Stale Pool
    write_requests,    # (N_write, 512) uint8 - 写请求序列
    selector,          # 选择策略函数
    encoder,           # 编码器函数
    rng,               # numpy 随机生成器
    init_hook,         # 训练初始化钩子 (LearnWD 的首次 train)
    retrain_hook,      # 重训练钩子 (LearnWD 每 RETRAIN_INTERVAL 触发)
    on_write_fn,       # 写后回调 (LearnWD 的 invalidate)
    ecc_level=-1,      # -1=无ECC; ≥0 则 WD 错误超过该值时计一次 VnR
    retrain_interval,  # 覆盖 config.py 中的 RETRAIN_INTERVAL
) -> SimResult
```

每步写操作流程

```
for new_block in write_requests:
    1. stale_addr = selector(new_block, ...)      # 选 stale block
    2. encoded, mask = encoder(new_block, stale)  # 编码
    3. wd_prone = compute_wd_prone(...)           # WD-prone 计数
    4. wd_errors = simulate_wd_errors(...)        # 采样 WD 错误
    5. vnr = 1 if ECC且errors>ecc_level else 0   # VnR 判断
    6. 记录 latency / energy / cost
    7. 更新 stale_memory[stale_addr] = encoded    # 物理写入
    8. stale_table: delete(stale_addr) → insert  # 重置聚类信息
    9. on_write_fn(stale_addr)                   # 通知 LearnWD 失效
    10. overwrite_counter += 1
        if counter >= RETRAIN_INTERVAL: retrain_hook(...)
```

`SimResult.summary()` 返回字典

```
{
    "mean_wd_prone": float,      # 每次写的平均 WD-prone 计数
    "mean_wd_errors": float,     # 每次写的平均 WD 错误数 (主要指标)
    "total_wd_errors": int,      # 总 WD 错误数
    "mean_latency_ns": float,    # 平均写延迟 / ns
    "mean_energy_nJ": float,     # 平均写能耗 / nJ
    "mean_write_cost": float,    # 平均写代价 (2R+S)
    "total_vnr": int,           # 总 VnR 次数 (Exp#6 专用)
    "n_retrains": int,          # 重训练次数
    "n_writes": int,            # 总写操作数
}
```

9. data_loader.py — 实验数据加载

功能：为 `run.py` / `run_sensitivity.py` 提供按名称加载数据集的接口，支持 MNIST、多种合成数据和原始文件。

`load_dataset(name, ...) → (stale_blocks, write_requests)`

```
stale_blocks, write_requests = load_dataset("mnist")
# stale_blocks: (50_000, 512) uint8
# write_requests: (50_000, 512) uint8
```

支持的 `name` 值：

name	说明
<code>"mnist"</code>	MNIST 手写数字 (sklearn openml 下载)
<code>"synthetic"</code> / <code>"synthetic_random"</code>	iID 0.5 密度随机位
<code>"synthetic_sparse"</code>	0.05 密度 (模拟稀疏数据)
<code>"synthetic_dense"</code>	0.90 密度 (模拟密集数据)
<code>"synthetic_alt"</code>	严格交替 0101... 模式
<code>"synthetic_corr"</code>	局部相关位 (Markov 链, 平均游程 32)
<code>"raw:<path>"</code>	直接读取本地文件字节

`bytes_to_blocks(data, block_bits, n_blocks)` — 核心转换函数

```
bits = np.unpackbits(np.frombuffer(data, dtype=np.uint8)) # 字节展开为 bit 数组
bits = tile_to(bits, n_blocks * block_bits)             # 若不够则循环填充
return bits.reshape(n_blocks, block_bits)               # 切成 block
```

所有数据来源（数值矩阵、图像字节、文本 UTF-8）都最终通过此函数转换为 (N, 512) 的 bit 矩阵。

10. download_datasets.py — 全量数据集下载/生成

功能：从各个数据源（UCI / sklearn / Kaggle / HuggingFace / 本地 zip）下载原始数据，转换为

(100_000, 512) uint8 的 .npy 文件保存到 `datasets/`。

REGISTRY 结构

```
REGISTRY = {
    "AMZ": {"name": "...", "type": "numerical", "loader": _load_AMZ},
    ...
    "KNS": {"name": "Kensho Wikipedia", "type": "textual", "loader": _load_KNS},
}
```

每个 dataset 有三个字段：`name`（显示名）、`type`（numerical/multimedia/textual）、`loader`（加载函数）。

新增数据集的步骤

1. 在文件末尾（`_load_INB` 之后）新增加载函数 `_load_XXX()`：

```
python def _load_XXX(): # 加载原始数据... return _text_to_blocks(texts) # 文本类 # 或 return
_numeric_to_blocks(X) # 数值类 # 或 return _bytes_to_blocks(raw) # 字节流类
```

2. 在 `REGISTRY` 字典中添加条目：

```
python "XXX": {"name": "My Dataset", "type": "textual", "loader": _load_XXX},
```

3. 在 `ORDER` 列表末尾添加 `"XXX"`

4. 运行生成：

```
bash .venv/bin/python3 -c " from download_datasets import _load_XXX, OUT_DIR import numpy
as np blocks = _load_XXX() np.save(OUT_DIR / 'XXX.npy', blocks) print('density:',
blocks.mean()) "
```

三种数据转换辅助函数

```
_bytes_to_blocks(raw: bytes) # 原始字节 → blocks (图像 JPEG, CSV 文本等)
_numeric_to_blocks(X: array) # 数值矩阵 → float32 bytes → blocks
_text_to_blocks(texts: list[str]) # 文本列表 → UTF-8 encode → bytes → blocks
```

`_load_KNS()` — Kensho Wikimedia 加载

直接从 `datasets/kensho.zip` 流式读取 `link_annotated_text.jsonl`，无需解压（节省 25 GB 磁盘空间）：

```

with zipfile.ZipFile("datasets/kensho.zip") as z:
    with z.open("link_annotated_text.jsonl") as f:
        for raw_line in f:
            obj = json.loads(raw_line)
            for sec in obj["sections"]:
                texts.append(sec["text"])
            if len(texts) >= 5000: break

```

11. experiments.py — 灵敏度实验实现

功能： `run_sensitivity.py` 调用的所有灵敏度实验函数库。

实验参数常量

```

ECC_LEVELS      = [0, 1, 2, 4, 8]
K_VALUES        = [2, 4, 8, 16, 32]
H_VALUES        = [0, 1, 2, 4, 8, 16]
RETRAIN_INTERVALS = [5_000, 10_000, 20_000, 30_000, 50_000]
BLOCK_SIZES     = [("64B", 512), ("256B", 2048), ("1KB", 8192), ("4KB", 32768)]
CLUSTER_ALGOS   = ["kmeans", "gmm", "birch"]
SWITCH_DATASETS = ["mnist", "synthetic_random", "synthetic_alt",
                   "synthetic_dense", "synthetic_corr", "synthetic_sparse"]

```

各实验函数签名

```

run_exp6(stale_blocks, write_requests, seed)  list[dict] # ECC × Selector
run_exp7_k(stale_blocks, write_requests, seed) list[dict] # k 灵敏度
run_exp8_h(stale_blocks, write_requests, seed) list[dict] # h 灵敏度
run_exp9_retrain(stale_blocks, write_requests, seed) list[dict] # retrain interval
run_exp10_blocksize(dataset_name, seed)      list[dict] # block 大小
run_exp11_algo(stale_blocks, write_requests, seed) list[dict] # 聚类算法
run_exp12_switch(seed)                       list[dict] # 数据集切换

```

每个函数返回 `list[dict]`，每个 dict 是一行实验结果（字段根据实验不同而异）。

12. run.py — Exp#1-5 入口

功能：在单个数据集上运行 8 种配置（RandSel/LearnWD × DCW/DIN/DMPart/MinWD），打印归一化对比表。

用法

```

# 语法
.venv/bin/python3 run.py [dataset] [seed] [encoders]

# 示例
.venv/bin/python3 run.py mnist 0 all          # MNIST, seed=0, 全部编码器
.venv/bin/python3 run.py mnist 0 dcw,minwd    # 只跑 DCW + MinWD
.venv/bin/python3 run.py synthetic 0 all      # 随机合成数据
.venv/bin/python3 run.py raw:mydata.bin 0 all # 本地原始文件

```

输出

Encoder+Selector	WD errors	WD prone	Write cost	Energy (nJ)	Latency (ns)
DCW + RandSel	1.0000	1.0000	1.0000	1.0000	1.0000
DCW + LearnWD	0.5429	0.5428	0.6969	0.8354	0.8374
...					
(归一化到 DCW + RandSel = 1.000)					

13. run_all.py — 全量 15/16 数据集实验

功能：加载 `datasets/*.npy`，在所有数据集上运行 8 配置实验，输出 Fig 13-16 表格，结果保存到 `results_all.json`。

用法

```
# 语法
.venv/bin/python3 run_all.py [seed] [--fast] [--download]

# 示例
.venv/bin/python3 run_all.py 0                # 标准运行 (~40分钟)
.venv/bin/python3 run_all.py 0 --fast          # 快速测试 (每个数据集只用10k写请求)
.venv/bin/python3 run_all.py 0 --download      # 先重新下载数据集再跑
```

DS_ORDER — 控制运行哪些数据集

`run_all.py` 第 21 行：

```
DS_ORDER = ["AMZ", "BKS", "CES", "SPM", "WKM",
            "FMN", "MNI", "FRT", "RIE",
            "BEP", "TSC", "HNT", "IMD", "RTC", "INB"]
```

如要加入 KNS，在此列表末尾添加 `"KNS"`（前提是 `datasets/KNS.npy` 已存在且 `manifest.json` 中有 `KNS` 条目）。

增量保存

每完成一个数据集就写一次 `results_all.json` (crash-safe)，中断后不会丢失已计算的结果。

14. run_sensitivity.py — Exp#6-12 灵敏度实验入口

功能：调用 `experiments.py` 中的函数，运行 7 项灵敏度分析，打印结果表格。

用法

```
# 语法
.venv/bin/python3 run_sensitivity.py [dataset] [seed] [experiments]

# 示例
.venv/bin/python3 run_sensitivity.py mnist 0 all      # 全部 Exp#6-12
.venv/bin/python3 run_sensitivity.py mnist 0 7,8     # 只跑 Exp#7 和 #8
.venv/bin/python3 run_sensitivity.py mnist 0 6       # 只跑 ECC 实验
```

注意：Exp#10 (block 大小) 和 Exp#12 (数据集切换) 在函数内部自行加载数据，不受 `[dataset]` 参数影响。

15. 如何运行各实验

所有命令在 `/Users/pudding/Desktop/learnWD/` 目录下运行。

前提：准备虚拟环境

```
cd /Users/pudding/Desktop/learnWD
# 确认 .venv 存在且依赖已安装
.venv/bin/python3 -c "import numpy, sklearn; print('OK')"
```

准备数据集（首次运行）

```
# 下载/生成全部 16 个数据集（约需几分钟到数小时，取决于网速）
.venv/bin/python3 download_datasets.py

# 仅重新生成 KNS（本地 zip, 0.2 秒）
.venv/bin/python3 -c "
from download_datasets import _load_KNS, OUT_DIR
import numpy as np, json
from pathlib import Path
blocks = _load_KNS()
np.save(OUT_DIR / 'KNS.npy', blocks)
manifest = json.loads(Path('datasets/manifest.json').read_text())
manifest['KNS'] = {'name': 'Kensho Wikipedia', 'type': 'textual',
                  'source': 'real', 'bit_density': round(float(blocks.mean()), 3)}
Path('datasets/manifest.json').write_text(json.dumps(manifest, indent=2))
print('KNS density:', blocks.mean())
"
```

Exp#1-5: 8 配置单数据集对比

```
# MNIST（最常用）
.venv/bin/python3 run.py mnist 0 all

# 其他数据集
.venv/bin/python3 run.py synthetic_sparse 0 all
.venv/bin/python3 run.py raw:datasets/KNS.npy 0 all # 用 KNS 作为输入（注意需 raw: 前缀）
```

Exp#6-12: 灵敏度分析

```
# 全部灵敏度实验 (约 70 分钟)
.venv/bin/python3 run_sensitivity.py mnist 0 all

# 单项实验
.venv/bin/python3 run_sensitivity.py mnist 0 6      # ECC 灵敏度
.venv/bin/python3 run_sensitivity.py mnist 0 7      # k 灵敏度
.venv/bin/python3 run_sensitivity.py mnist 0 8      # h 灵敏度
.venv/bin/python3 run_sensitivity.py mnist 0 9      # retrain 间隔
.venv/bin/python3 run_sensitivity.py mnist 0 10     # block 大小 (约 15 分钟)
.venv/bin/python3 run_sensitivity.py mnist 0 11     # 聚类算法 (约 15 分钟, BIRCH 慢)
.venv/bin/python3 run_sensitivity.py mnist 0 12     # 数据集切换

# 组合运行
.venv/bin/python3 run_sensitivity.py mnist 0 7,8,9
```

全量 15 数据集主实验

```
# 标准运行 (约 40 分钟)
.venv/bin/python3 run_all.py 0

# 快速冒烟测试 (每个数据集 10k 写, 约 3 分钟)
.venv/bin/python3 run_all.py 0 --fast

# 保存到日志并保持输出
PYTHONUNBUFFERED=1 .venv/bin/python3 -u run_all.py 0 > results.log 2>&1 &
tail -f results.log # 实时查看进度
```

查看 KNS 单独实验结果

```
PYTHONUNBUFFERED=1 .venv/bin/python3 -u run.py raw:datasets/KNS.npy 0 all
```

16. 修改代码后如何重新运行

A. 修改了 `config.py` (PCM 参数或 LearnWD 超参数)

影响: 所有实验结果均需重跑, 因为能耗/延迟常数或 k/h 默认值改变。

```
# 重跑全量实验
.venv/bin/python3 run_all.py 0

# 重跑灵敏度实验
.venv/bin/python3 run_sensitivity.py mnist 0 all

# 重跑单数据集对比
.venv/bin/python3 run.py mnist 0 all
```

B. 修改了 `pcm_sim.py` (物理模型)

影响: 所有写操作的 WD-prone 计算或能耗/延迟模型改变, 需重跑全量实验。同 A。

C. 修改了 `encoding.py` (某种编码器)

影响: 只有用到该编码器的实验受影响。

```
# 只重跑受影响的编码器 (如修改了 dcw_encode)
.venv/bin/python3 run.py mnist 0 dcw      # 只跑 dcw
.venv/bin/python3 run_sensitivity.py mnist 0 all # 灵敏度实验默认用 dcw
```

如果要在 `run_all.py` 里只重跑单个编码器：手动修改 `ENCODERS` 字典后运行，或写小脚本。

D. 修改了 `learnwd.py` (LearnWD 模型逻辑)

影响：所有用 `LearnWD` 的配置 (`LW+DCW` , `LW+DIN` , `LW+DMP` , `LW+MWD`) 。

```
# 快速验证：单数据集 2 配置
.venv/bin/python3 run.py mnist 0 dcw      # 只看 LearnWD+DCW 和 RandSel+DCW

# 重跑全量
.venv/bin/python3 run_all.py 0
```

E. 修改了 `download_datasets.py` (某个 loader)

只需重新生成对应的 .npy 文件：

```
# 强制重新生成所有数据集 (--force 忽略缓存)
.venv/bin/python3 download_datasets.py --force

# 只重新生成 KNS
.venv/bin/python3 -c "
from download_datasets import _load_KNS, OUT_DIR
import numpy as np
np.save(OUT_DIR / 'KNS.npy', _load_KNS())
"

# 然后重跑实验
.venv/bin/python3 run_all.py 0 # 或 run.py
```

F. 新增一个数据集并加入全量实验

1. 在 `download_datasets.py` 添加 `_load_XXX()` 函数、REGISTRY 条目、ORDER 列表
2. 生成 .npy 文件 (见第 10 节)
3. 更新 `manifest.json` (见第 10 节中的命令)
4. 在 `run_all.py` 的 `DS_ORDER` 中添加 `"XXX"`
5. 运行：

```
bash .venv/bin/python3 run_all.py 0
```

G. 修改了灵敏度实验参数 (`experiments.py` 中的常量列表)

直接修改对应常量后重跑：

```
# 例如修改了 K_VALUES 后
.venv/bin/python3 run_sensitivity.py mnist 0 7

# 修改了 BLOCK_SIZES 后
.venv/bin/python3 run_sensitivity.py mnist 0 10
```

H. 只想验证某次修改是否正确（快速冒烟测试）

```
# 5k 写请求快速测试 (约 30 秒)
.venv/bin/python3 -c "
from data_loader import load_dataset
from encoding import dcw_encode
from selector import randssel, make_learnwd_selector
from learnwd import LearnWDModel
from simulation import run_simulation
import numpy as np

stale, writes = load_dataset('mnist')
writes = writes[:5000] # 只用 5000 次写请求

# RandSel
r1 = run_simulation(stale, writes, randssel, dcw_encode, np.random.default_rng(0))
print('RandSel WD:', r1.summary()['mean_wd_errors'])

# LearnWD
model = LearnWDModel()
sel = make_learnwd_selector(model)
r2 = run_simulation(stale, writes, sel, dcw_encode, np.random.default_rng(0),
                    init_hook=lambda sm,st,r: model.train(sm,st),
                    retrain_hook=lambda sm,st,r: model.train(sm,st),
                    on_write_fn=model.invalidate)
print('LearnWD WD:', r2.summary()['mean_wd_errors'])
print('改善:', 1 - r2.summary()['mean_wd_errors'] / r1.summary()['mean_wd_errors'])
"
```

最后更新: 2026-05-24